

Lab06-CopyDirWithMultiProcess

目标

写一个多进程复制程序，实现文件夹的复制，并与单进程的复制程序做效率比较

工具

GCC编译器

代码块

```
1 sudo apt-get install build-essential
```

IDE

1. (推荐)Code::Block

代码块

```
1 sudo apt-get install codeblocks
```

获得程序的执行时间：time命令

代码块

```
1 time pwd
```

比较目录是否相同：diff命令

代码块

```
1 diff -r DirA DirB
```

相关函数

创建进程，以及执行程序相关的函数：

execv

代码块

```
int execv(const char *path, char *const argv[]);
```

- **参数：**

- `path`：可执行文件的完整路径。
- `argv[]`：字符串数组，存放命令行参数（`argv[0]` 通常是程序名，最后一个元素为 `NULL`）。

- **示例：**执行 `ls -l /home`

代码块

```
1 char *argv[] = {"ls", "-l", "/home", NULL};
2 execv("/bin/ls", argv);
```

execvp

代码块

```
1 int execvp(const char *file, char *const argv[]);
```

- **参数：**

- `file`：可执行文件的文件名（自动搜索 `PATH`）。
- `argv[]`：同 `execv` 的参数数组。

- **示例：**执行 `ls -l /home`（无需完整路径）

代码块

```
1 char *argv[] = {"ls", "-l", "/home", NULL};
2 execvp("ls", argv);
```

实验流程

1. 先理解并测试单进程复制目录程序

代码块

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/wait.h>
5
```

```

6 // 单进程复制: 直接用cp -r复制整个源文件夹到目标
7 int main(int argc, char *argv[]) {
8     if (argc != 3) {
9         fprintf(stderr, "用法: %s <源文件夹> <目标文件夹>\n", argv[0]);
10        return 1;
11    }
12
13    // 创建子进程执行cp命令
14    pid_t pid = fork();
15    if (pid == -1) {
16        perror("fork失败");
17        return 1;
18    }
19
20    if (pid == 0) { // 子进程: 执行cp -r 源 目标
21        char *const args[] = {"cp", "-r", argv[1], argv[2], NULL}; // 参数列表以
NULL结尾
22        execvp("cp", args); // 执行cp命令
23        perror("execvp失败"); // 若execvp返回, 说明执行失败
24        exit(1);
25    } else { // 父进程: 等待子进程完成
26        int status;
27        waitpid(pid, &status, 0);
28        if (WIFEXITED(status) && WEXITSTATUS(status) == 0) {
29            printf("单进程复制完成\n");
30        } else {
31            fprintf(stderr, "单进程复制失败\n");
32            return 1;
33        }
34    }
35
36    return 0;
37 }

```

2. 再理解并测试多进程打印目录程序

代码块

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/wait.h>
5
6 int main() {
7     const int num_processes = 3; // 3个进程
8     pid_t pids[num_processes];

```

```

9
10 // 创建3个子进程
11 for (int i = 0; i < num_processes; i++) {
12     pids[i] = fork();
13     if (pids[i] == -1) {
14         perror("fork失败");
15         exit(1);
16     } else if (pids[i] == 0) { // 子进程逻辑
17         printf("\n----- 进程 %d (PID: %d) 执行ls -----\\n", i+1, getpid());
18         // 执行ls -l命令
19         char *const args[] = {"ls", "-l", NULL};
20         execvp("ls", args);
21         // 若execvp返回, 说明执行失败
22         perror("execvp执行ls失败");
23         exit(1);
24     }
25 }
26
27 // 父进程等待所有子进程完成
28 for (int i = 0; i < num_processes; i++) {
29     waitpid(pids[i], NULL, 0);
30     printf("\\n进程 %d (PID: %d) 执行完毕\\n", i+1, pids[i]);
31 }
32
33 printf("\\n所有进程执行完成\\n");
34 return 0;
35 }

```

3. 结合上面的两个示例程序，实现多进程复制目录的程序

- 将linux内核源码下的include文件夹，复制为include-backup-学号
- 根据include下面的文件及文件夹数量新建进程，每个文件或文件夹启动一个进程进行复制

4. 比较单进程复制和多进程复制的时间消耗

5. 验证多进程复制的结果与原始目录相同

6. 根据上述过程，撰写实验报告